

HACK FOR HUMANITY HARARE 2026

BEGINNER'S BUILD-DAY GUIDE

From opening your laptop at 08:30 to a working Gemini-powered prototype
and a submission-ready demo

What this guide is for

This guide is written for a participant who may be completely new to APIs, virtual environments, VS Code, requirements files, Streamlit, GitHub and Gemini integration. It follows a simple local Windows workflow and explains not only what to type, but why each step exists.

Saturday, 19 September 2026 | CZI, Harare | 08:30-17:30

Official challenge: Best Use of the Google Gemini API

Prepared as a practical participant handbook based on the actual setup difficulties encountered during the Harare hackathon build.

How to use this guide

You do not need to read this document from cover to cover before coding. Use the Day Map first, then follow the numbered steps in order. The guide assumes Windows 10/11 because that is the most common path used in the worked example. Mac and Linux commands are included in an appendix.

Beginner rule

Do not try to understand everything before starting. Complete one small step, confirm that it works, and only then move to the next step. A working simple prototype is far more valuable than an ambitious project that never runs.

Guide contents

1. The challenge in plain English
2. The 08:30-17:30 beginner day map
3. The recommended coding environment
4. Laptop setup: Python, VS Code and the Python extension
5. Creating the project folder and files
6. Creating and activating a virtual environment
7. Installing dependencies with requirements.txt
8. Creating and testing a first Streamlit web app
9. Creating a Gemini API key without accidentally buying a paid plan
10. Protecting the API key
11. Testing Gemini from Python
12. Connecting Gemini to Streamlit
13. Turning the starter into your actual hack
14. Testing, error handling and the 503/high-demand problem
15. GitHub, README and submission preparation
16. Demo preparation and final submission checklist
- Appendix A. Command cheat sheet
- Appendix B. File-by-file glossary
- Appendix C. Mac/Linux commands
- Appendix D. Troubleshooting table

1. The challenge in plain English

The hackathon challenge is not simply to ask Gemini questions or use Gemini to help write code. The project should contain an actual Gemini API integration inside the prototype. By demo time, a team should be able to show the human problem, the intended user, what was built during the day, exactly where Gemini is used, what the Gemini capability adds, what works, and what limitations or safeguards were considered.

The simplest mental model

Your app collects something from the user -> your Python code sends relevant information to Gemini through an API -> Gemini returns something useful -> your app displays or uses that response.

For a beginner, a good first architecture is:

```
USER -> STREAMLIT WEB PAGE -> PYTHON -> GEMINI API -> PYTHON -> STREAMLIT RESULT
```

What the judges care about

The hackathon briefing states that MLH uses four equally weighted areas: Technology, Design, Completion and Learning. This means a small prototype that works and is easy to explain can perform better than a very large unfinished system.

Area	Beginner interpretation
Technology	Does the project genuinely use the Gemini API and does the technical implementation work?
Design	Can a real user understand what to do and why the output is useful?
Completion	Can you demonstrate the core flow successfully rather than only describe it?
Learning	Can you explain what you tried, what failed, what you changed and what you learned?

Your minimum viable hack

- **One human problem.** Do not solve five problems.
- **One primary user.** Know exactly who uses the prototype.
- **One meaningful Gemini capability.** Use the API where it genuinely helps.
- **One main workflow.** Input -> Gemini-assisted processing -> useful output.
- **One working demo.** You should be able to repeat it in front of judges.

2. The 08:30-17:30 beginner day map

The schedule below is deliberately conservative. A beginner should budget real time for setup. The goal is to have a working end-to-end prototype by early afternoon so the final hours are available for testing, documentation and demo preparation.

Time	Focus	What you do	Milestone
08:30-08:45	Choose the problem	Write one sentence: user + problem + how Gemini helps.	No coding yet.
08:45-09:30	Set up the laptop	Confirm/install Python, VS Code, Python extension and Gemini API access.	Do not lose hours fighting tools silently; ask for help.
09:30-09:50	Create project	Create folder, files, virtual environment and install packages.	Project skeleton exists.
09:50-10:05	Hello World	Run a tiny Streamlit app.	Browser opens on localhost.
10:05-10:25	Test Gemini	Run one Python request to Gemini.	You know the API key and SDK work.
10:25-12:00	Build core workflow	Build one input -> Gemini -> output flow.	First working MVP.
12:00-12:15	Checkpoint	Demo it to a teammate exactly as a judge would see it.	Find blockers early.
12:15-14:00	Make it useful	Add the domain logic, prompts, simple validation and one strong scenario.	Core value visible.
14:00-15:00	Improve UX and safeguards	Labels, instructions, loading messages, limitations, API error handling.	Demo no longer feels fragile.

15:00-16:00	Test and document	Test normal and failure cases; prepare README; clean repo.	Repeatable build.
16:00-16:45	Submission package	Repo link, description, technologies, team, challenge selection.	Submission-ready.
16:45-17:15	Rehearse demo	Practice a 2-3 minute demonstration twice.	Everyone knows who says what.
17:15-17:30	Final check	Submit before the organizer-announced deadline; keep backup screenshots.	Done.

If you arrive without Python or VS Code

Do not panic. The first hour can still be used to install them. Streamlit also documents cloud/browser options, including GitHub Codespaces, but for this guide the primary route is a local Windows setup because it gives you a normal project folder, terminal and files you can understand and reuse later.

3. The recommended coding environment

For a beginner Python team, the easiest environment to understand is a normal project folder opened in Visual Studio Code (VS Code). VS Code is where you create and edit Python files, Markdown files, requirements.txt and configuration files. Its integrated terminal lets you run the commands without switching constantly between applications.

Tool	What it does	Why you need it
Python	Runs your application code.	The project itself is written in Python.
VS Code	Editor and project workspace.	Where you create app.py, requirements.txt, README.md and other files.
VS Code Python extension	Adds Python language support and interpreter selection.	Makes Python development easier and helps VS Code recognise .venv.
Terminal / Command Prompt	Runs setup and application commands.	Used for venv, pip and streamlit commands.
Streamlit	Turns Python scripts into browser-based apps.	Lets a beginner build a usable interface without writing a full frontend.
Google AI Studio	Manages Gemini projects and API keys; also provides model tools.	Where you obtain the API credential used by your app.
google-gemai	Official Python SDK for Gemini.	Lets Python call Gemini without manually constructing HTTP requests.
GitHub	Stores and versions source code.	Useful for collaboration, judging and final submission.

What the environment looks like

```
VS CODE
|-- Explorer (your files)
|   |-- app.py
|   |-- requirements.txt
|   |-- README.md
|   |-- .gitignore
```

```
|  `-- .env.example
|
|-- Editor (where you write code)
|
|-- Terminal
|   |-- python -m venv .venv
|   |-- pip install -r requirements.txt
|   |-- streamlit run app.py
```

Browser -> shows the Streamlit app

Google AI Studio -> provides/manages the Gemini API key

GitHub -> stores the project repository

4. 08:45 - Set up Python, VS Code and the Python extension

STEP 4.1 Check whether Python is already installed

Open Command Prompt and type:

```
python --version
```

Then type:

```
pip --version
```

If both commands return version numbers, you are ready. Streamlit's current documentation supports Python 3.10 through 3.14. If Windows says Python was not found, install Python before continuing.

STEP 4.2 Install Python on Windows if it is missing

1. Go to the official Python website and download the current Windows 64-bit installer.
2. Run the installer.
3. On the first installation screen, tick 'Add python.exe to PATH'. This is important because it lets Command Prompt recognise the python command.
4. If this is your own laptop and you have administrator rights, using admin privileges for the launcher is fine but not essential.
5. Finish the installation, close Command Prompt, open a new Command Prompt and repeat `python --version` and `pip --version`.

Official Python downloads: <https://www.python.org/downloads/windows/> — Use the official site rather than an unknown third-party installer.

STEP 4.3 Install VS Code

6. Download Visual Studio Code from Microsoft's official VS Code website.
7. Install it using the default options.
8. Open VS Code.
9. Click the Extensions icon on the left sidebar.
10. Search for 'Python' published by Microsoft and install it.

VS Code: <https://code.visualstudio.com/> — The recommended editor used throughout this guide.

VS Code Python tutorial: <https://code.visualstudio.com/docs/python/python-tutorial> — Official explanation of Python, environments and workspaces.

5. 09:30 - Create the project folder and files

STEP 5.1 Create a project folder

Create a normal folder somewhere easy to find, for example on your Desktop:

```
hack-for-humanity-gemini-app
```

Then open VS Code and choose:

```
File -> Open Folder -> select hack-for-humanity-gemini-app
```

Important concept: a VS Code workspace

When VS Code opens a folder, that folder becomes your project workspace. Every file you create for the hack should normally live inside this folder.

STEP 5.2 Create the starter files

In the VS Code Explorer panel, click the New File icon and create the following files one by one. Type the names exactly, including dots and extensions.

File	What to put there	Why it exists
app.py	Your Streamlit + Gemini Python code.	This is the main application.
requirements.txt	Names of Python packages.	Lets pip recreate the project's dependencies.
README.md	Project explanation and run instructions.	GitHub displays it as the repository landing page.
.gitignore	Names of files/folders Git should ignore.	Prevents .env and secret files from being uploaded.
.env.example	A fake/example environment-variable template.	Shows collaborators what secret variables are required without exposing the real key.
test_gemini.py	A tiny one-request API test.	Lets you test Gemini independently before mixing it into the full app.

Exactly what to type into the first files

requirements.txt

```
streamlit
google-gemai
python-dotenv
```

.gitignore

```
.venv/
.env
__pycache__/
*.pyc
```

.env.example

```
GEMINI_API_KEY=replace_with_your_key
```

Leave README.md mostly empty for now. Add a title such as '# My Hack for Humanity Project'. You will complete it later once the project actually works.

6. 09:35 - Create and activate a virtual environment

A virtual environment is a private Python toolbox for this project. It keeps this hackathon's packages separate from other Python projects on your laptop.

STEP 6.1 Open the VS Code terminal

11. In VS Code choose Terminal -> New Terminal.
12. Check that the terminal path ends in your project folder.

STEP 6.2 Create .venv

```
python -m venv .venv
```

After a short wait, a new .venv folder should appear in VS Code. Do not edit its contents manually.

STEP 6.3 Activate .venv

Windows Command Prompt:

```
.venv\Scripts\activate
```

Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

When activation works, you should see '(.venv)' at the beginning of the terminal prompt.

If activation is confusing

You can also use VS Code's Command Palette (Ctrl+Shift+P) -> 'Python: Create Environment' -> Venv. VS Code can create and select the environment for you.

7. 09:40 - Install the project dependencies

With (.venv) visible in the terminal, run:

```
pip install -r requirements.txt
```

pip reads requirements.txt, downloads Streamlit, the Gemini SDK and python-dotenv, then installs their supporting libraries. A large amount of scrolling text is normal. Wait until the terminal returns to the prompt.

What 'Successfully installed' means

It means the packages are now inside this project's .venv. You do not need to understand every package that pip prints. Dependencies often have dependencies of their own.

8. 09:50 - Create and run your first Streamlit app

STEP 8.1 Write the smallest possible app

Open app.py and type:

```
import streamlit as st

st.title("My Hack for Humanity App")
st.write("Hello! My app is running.")
```

STEP 8.2 Run it

Save app.py, then run:

```
streamlit run app.py
```

A browser tab should open automatically. If it does not, copy the Local URL shown in the terminal, usually `http://localhost:8501`, and open it in your browser.

What localhost means

localhost means the application is running on your own laptop. It is not yet a public website. That is perfectly fine while building and testing.

To stop Streamlit, return to the terminal and press:

```
Ctrl + C
```

9. 10:05 - Create a Gemini API key

This is the step that often confuses beginners because Google AI Studio can show model subscriptions, pay-per-request options and free-tier API projects in the same interface. You need an API key for the Gemini API. You do not automatically need to purchase a monthly subscription.

STEP 9.1 Open Google AI Studio

Google AI Studio: <https://aistudio.google.com/> — Sign in with your Google account.

STEP 9.2 If the Playground asks you to upgrade

Do not assume you are in the wrong place. The Playground may be gated for your account or selected model. Instead, use the management dashboard:

```
Google AI Studio -> Dashboard -> API Keys
```

STEP 9.3 Check your project and billing tier

- Look for a project such as 'Default Gemini Project'.
- Look at the Billing Tier column.
- If the project is marked Free tier, you can begin with that tier subject to Google's model and rate limits.
- Do not click 'Set up billing' merely because the link is visible; it is the upgrade route, not a requirement for an already-active free-tier project.

STEP 9.4 Create the API key

13. Click 'Create API key'.
14. Choose the project you intend to use, for example Default Gemini Project.
15. Create the key.
16. Copy it temporarily.
17. Never post the full key in WhatsApp, screenshots, public GitHub repositories or hackathon slides.

Google's current documentation states that new API keys created in Google AI Studio are authorization keys and that every key is associated with a Google Cloud project.

Official Gemini API key guide: <https://ai.google.dev/gemini-api/docs/api-key> — Current instructions for projects, keys and secure environment setup.

Gemini billing guide: <https://ai.google.dev/gemini-api/docs/billing> — Explains Free Tier and paid tiers.

10. 10:15 - Protect the Gemini API key

For a quick hackathon prototype there are two beginner-friendly approaches. The first is easiest for a local demo. The second is better as the project matures.

Option A - easiest for the hackathon: enter the key in the app

Streamlit can display a password-style input box. The key is entered only when the app is running locally and is not written into app.py.

```
api_key = st.sidebar.text_input(
    "Gemini API key",
    type="password"
)
```

Do not hard-code the key

Never write something like `client = genai.Client(api_key="Alza...your-real-key...")` in code that will be uploaded to GitHub.

Option B - better local development: use an environment variable

Google recommends environment variables. On Windows you can create a user environment variable called GEMINI_API_KEY and reopen the terminal. The Gemini client can then detect it automatically.

```
GEMINI_API_KEY = your secret value
```

A local .env file is also common when using python-dotenv. If you create .env, make sure .gitignore contains '.env' before you ever push the project to GitHub.

11. 10:20 - Test Gemini separately before adding it to the app

This is one of the best debugging habits for a beginner. If a tiny Gemini test works but the Streamlit app does not, you know the problem is in your application rather than the API credential or SDK.

STEP 11.1 Create test_gemini.py

Use a simple version like this:

```
from google import genai

# If GEMINI_API_KEY is set as an environment variable:
client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3.6-flash",
    input="Explain AI governance in one simple sentence."
)

print(interaction.output_text)
```

The exact model name can change over time. Use a model currently supported by your project or the model recommended by the hackathon organisers. The example uses a current Flash model because Flash models are suited to fast interactive prototypes.

Run:

```
python test_gemini.py
```

If Gemini returns text, celebrate: your Python installation, virtual environment, SDK and API access are connected.

12. 10:25 - Connect Gemini to your Streamlit app

Now combine the two pieces you tested separately: the Streamlit interface and the Gemini API.

STEP 12.1 Replace app.py with a tiny Gemini starter

```
import streamlit as st
from google import genai

st.set_page_config(page_title="Hack for Humanity Starter")
st.title("Hack for Humanity - Gemini Starter")

api_key = st.sidebar.text_input("Gemini API key", type="password")

problem = st.text_area(
    "Describe the human problem you want help thinking through"
)

if st.button("Ask Gemini"):
    if not api_key:
        st.error("Please enter your Gemini API key.")
    elif not problem.strip():
        st.error("Please describe the problem.")
    else:
        client = genai.Client(api_key=api_key)

        with st.spinner("Gemini is working..."):
            interaction = client.interactions.create(
                model="gemini-3.6-flash",
                input=f'''
You are helping a hackathon team understand a human problem.

Problem:
{problem}'''
```

```

Return:
1. Who is affected
2. Why the problem matters
3. Three possible solution directions
4. Important risks or safeguards
'''

    )

    st.subheader("Gemini response")
    st.write(interaction.output_text)

```

STEP 12.2 Run the app again

```
streamlit run app.py
```

Enter the API key in the sidebar, describe a problem, click Ask Gemini and check that the response appears in the browser.

You have now built a real API integration

The Gemini model is no longer only helping you write code. Your own application is making a request to Gemini at runtime and using the returned result. That is the fundamental concept the challenge is asking teams to demonstrate.

13. 10:30-14:00 - Turn the starter into your actual hack

Do not keep adding features randomly. Use the problem structure from the hackathon briefing:

Problem -> User -> Evidence -> Existing Solution -> Gap -> Gemini Capability -> Prototype -> Impact -> Risks/Safeguards

Use this one-page design worksheet

Question	Team answer
Problem	What specific human problem are we addressing?
User	Who will actually use this prototype?
Evidence	What tells us this is a real problem?
Current approach	How is the problem handled now?
Gap	What is missing or inefficient?
Gemini role	What can Gemini do here that genuinely helps?
Input	What will the user provide to the app?
Output	What useful result will the app return?
Safeguard	What could go wrong, and how will the prototype limit or communicate that risk?
Demo	What exact 60-120 second user journey will we show?

Choose one Gemini capability, not every capability

- **Text reasoning / extraction.** Turn free text into an organised analysis.
- **Structured outputs.** Return predictable fields such as JSON when your application needs reliable structure.
- **Document understanding.** Analyse an uploaded document if documents are central to the problem.
- **Image understanding.** Analyse images when visual information is essential.
- **Function calling.** Use Gemini to select or call application functions when actions are necessary.
- **RAG / file search.** Retrieve from your own documents when the problem depends on a knowledge base.

The hackathon briefing explicitly says you do not need to use every Gemini capability. Choose what genuinely serves the problem.

14. Build incrementally: the beginner development loop

Use the following loop repeatedly throughout the day:

1. Make ONE small change.
2. Save the file.
3. Let Streamlit rerun (or rerun manually).
4. Test the feature.
5. If it breaks, fix that small change before adding another.
6. Commit to Git when the project reaches a working checkpoint.

A sensible feature order

18. Make the page open.
19. Make the user input work.
20. Make one Gemini request work.
21. Display the Gemini response.
22. Improve the prompt so the response serves the user's real problem.
23. Add simple validation: empty input, missing API key, unreasonable values.
24. Add a loading message.
25. Add a limitation/safeguard notice.
26. Only then consider extra features such as file upload, structured outputs, multiple steps or persistence.

15. 14:00-15:00 - Make the app survive normal failures

A hackathon demo becomes much safer when common failures are handled gracefully. One real example encountered during development was a Gemini 503 response stating that a Flash model was experiencing high demand.

The 503 / high-demand problem

What it means

HTTP 503 generally means the service is temporarily unavailable or overloaded. It does not necessarily mean your API key, code or account is wrong.

A simple beginner approach is to catch the exception and show a friendly message:

```
try:
    interaction = client.interactions.create(
        model="gemini-3.6-flash",
        input=prompt
    )
```

```

st.write(interaction.output_text)

except Exception as error:
    st.error(
        "Gemini is temporarily unavailable. "
        "Please wait a moment and try again."
    )
    st.caption(str(error))

```

Other failures to test

Test	Expected behaviour
No API key	App explains that a key is required instead of crashing.
Empty user input	App asks the user to provide input.
Gemini temporarily unavailable	Friendly retry message appears.
Very long response	Page remains readable.
Bad/irrelevant input	Prompt or interface guides user back to intended use.
Refresh browser	You understand what state is retained and what is lost.
Internet drops	Team knows the app requires connectivity and has screenshots/backup evidence.

16. 15:00-16:00 - Prepare GitHub and documentation

GitHub is useful because judges and collaborators can inspect what was built. The hackathon briefing also asks teams to distinguish what they built during the Hack Day from frameworks, libraries and AI-assisted/generated material.

Minimum repository structure

```

your-project/
|-- app.py
|-- requirements.txt
|-- README.md
|-- .gitignore
|-- .env.example
`-- optional assets / sample data

DO NOT upload:
|-- .env
|-- .venv/
`-- your real API key

```

What README.md should contain

27. Project name and one-sentence description.
28. Human problem and intended user.
29. What the team built during Hack Day.
30. Exactly how the Gemini API is used.
31. Technology stack.
32. Installation/run instructions.

33. Known limitations and safeguards.
34. Team members.
35. Any pre-existing research, libraries or assets clearly disclosed.

Example README skeleton

```
# Project Name

## Problem
Who is affected and what problem are we solving?

## Solution
What did we build?

## Gemini integration
Where does the Gemini API run and what does it enable?

## Tech stack
Python, Streamlit, google-genai, ...

## Run locally
1. python -m venv .venv
2. .venv\Scripts\activate
3. pip install -r requirements.txt
4. streamlit run app.py

## Safeguards and limitations
What should users not assume?

## Hack-Day build boundary
What was built today vs. pre-existing material?
```

17. 16:00-17:30 - Submission and demo preparation

The organisers indicated that every presenting team must submit through OrganizerHQ and select the relevant partner challenge. The exact submission instructions and deadline are provided during the event.

Submission information to have ready

- Project name.
- Short project description.
- Technologies used.
- Repository/project link.
- Team information.
- Challenge selection: Best Use of the Google Gemini API.

Your 2-3 minute demo structure

Time	What to show	What to say
0:00-0:20	Problem	Who is affected and why this matters.
0:20-0:40	User + input	What the user gives the application.
0:40-1:30	Live workflow	Show the actual working app, not only slides.

1:30-2:00	Gemini integration	Point to the exact step where your app calls Gemini and what Gemini adds.
2:00-2:30	Output / impact	Why the result is useful.
2:30-2:50	Risk / safeguard	What could go wrong and how you bounded it.
2:50-3:00	Learning	One thing that failed or changed and what the team learned.

The final-hour rule

Stop adding ambitious features. Rehearse the exact demo, test the path you will show, confirm the repository is accessible, and submit before the organiser-announced deadline.

18. One-page emergency quick start

If your team is behind schedule, use this sequence only:

#	Action
1	Create/open project folder in VS Code.
2	Check <code>`python --version`</code> . Install Python if missing and ensure PATH is enabled.
3	Create <code>`app.py`</code> , <code>`requirements.txt`</code> , <code>`gitignore`</code> .
4	Put <code>`streamlit`</code> and <code>`google-genai`</code> in requirements.txt.
5	Run <code>`python -m venv .venv`</code> .
6	Run <code>`.venv\Scripts\activate`</code> .
7	Run <code>`pip install -r requirements.txt`</code> .
8	Run a Streamlit Hello World.
9	AI Studio -> Dashboard -> API Keys -> create key in an available project.
10	Add password-style API key input to app.py.
11	Make one Gemini API request from the app.
12	Test the exact demo path twice.
13	Push clean source files to GitHub; never push the real key or .venv.
14	Complete README and OrganizerHQ submission.

Appendix A. Command cheat sheet

Command	Meaning
<code>python --version</code>	Check whether Python is installed.
<code>pip --version</code>	Check whether pip is installed.
<code>python -m venv .venv</code>	Create a private Python environment for the project.
<code>.venv\Scripts\activate</code>	Activate .venv in Windows Command Prompt.
<code>pip install -r requirements.txt</code>	Install all packages listed in requirements.txt.
<code>streamlit hello</code>	Test whether Streamlit works.
<code>streamlit run app.py</code>	Start the Streamlit application.
<code>Ctrl + C</code>	Stop the running Streamlit server in the terminal.
<code>deactivate</code>	Leave the virtual environment.
<code>pip freeze > requirements.txt</code>	Generate a requirements file from currently installed packages; useful later, but may include more packages than a minimal hand-written file.

Appendix B. File-by-file glossary

Name	Type	Purpose
<code>app.py</code>	Python	Main application code. Streamlit UI, Gemini client, prompts and app logic live here.
<code>requirements.txt</code>	Text	Dependency list. Lets another person recreate the Python environment.
<code>README.md</code>	Markdown	Human-facing project documentation and setup instructions.
<code>.gitignore</code>	Text	Tells Git not to upload local environments, caches and secret files.
<code>.env</code>	Text (secret)	Optional local secrets. Must never be pushed publicly.
<code>.env.example</code>	Text	Safe template showing which environment variables are required.
<code>test_gemini.py</code>	Python	Small independent Gemini API test used for debugging.
<code>.venv/</code>	Folder	Local virtual environment created by Python. Do not upload it.
<code>__pycache__</code>	Folder	Python-generated cache. Do not edit or upload it.

Appendix C. Mac and Linux equivalents

The project files are the same. The main differences are virtual-environment activation and some shell conventions.

Task	Windows	macOS / Linux
------	---------	---------------

Create venv	<code>python -m venv .venv</code>	<code>python3 -m venv .venv</code> (or <code>python -m venv .venv</code>)
Activate venv	<code>.venv\Scripts\activate</code>	<code>source .venv/bin/activate</code>
Install packages	<code>pip install -r requirements.txt</code>	<code>pip install -r requirements.txt</code>
Run app	<code>streamlit run app.py</code>	<code>streamlit run app.py</code>
Stop app	Ctrl + C	Ctrl + C
Deactivate	<code>deactivate</code>	<code>deactivate</code>

Appendix D. Beginner troubleshooting table

Symptom	Likely cause	What to do
'python' is not recognized / Python was not found	Python is missing or not on PATH.	Install Python; tick Add python.exe to PATH; close and reopen terminal.
'pip' is not recognized	Python/pip PATH problem.	Try <code>'python -m pip --version'</code> ; reopen terminal after installation.
No (.venv) in prompt	Virtual environment is not active.	Run <code>.venv\Scripts\activate</code> in Command Prompt.
requirements.txt not found	Terminal is in the wrong folder.	Open the correct project folder in VS Code or <code>'cd'</code> into it.
streamlit is not recognized	Streamlit is not installed in active environment or wrong environment.	Activate .venv, run <code>pip install -r requirements.txt</code> ; try <code>'python -m streamlit run app.py'</code> .
Browser does not open	Streamlit still may be running.	Copy the Local URL from the terminal into your browser.
AI Studio asks for upgrade	Playground/account may be gated.	Use Dashboard -> API Keys; inspect project Billing Tier; create a key in an available project.
Gemini 401/403	API key or permission issue.	Check that the key belongs to the intended project and was copied correctly. Do not expose it while asking for help.
Gemini 429	Rate limit/quota reached.	Wait, reduce request frequency, check AI Studio Rate Limit/Usage. Do not create loops that repeatedly call Gemini.
Gemini 503 / high demand	Temporary service overload.	Wait and retry; add friendly error handling; consider an available supported fallback model.
Key accidentally pushed to GitHub	Credential exposure.	Revoke/replace the key immediately; remove secret from code and repository history as appropriate.
App works on one laptop only	Dependency or secret setup differs.	Use requirements.txt, README setup steps and .env.example; never copy .venv as the solution.

Appendix E. Suggested roles for a beginner team

Role	Primary responsibility
Problem lead	Keeps the team focused on the user and problem; prepares the pitch.

Builder	Edits app.py, runs the terminal commands and integrates Gemini.
Prompt/test lead	Tests prompts, edge cases and output quality; records failures and learnings.
UX/documentation lead	Improves labels/layout, writes README, prepares submission text and demo flow.

In a small team one person can hold more than one role. The point is to prevent everyone from editing the same thing while documentation and testing are forgotten.

Appendix F. Official resources to keep open in browser tabs

Google AI Studio: <https://aistudio.google.com/> — API keys, project management and model tools.

Gemini API key documentation: <https://ai.google.dev/gemini-api/docs/api-key> — Current key types and secure environment-variable setup.

Gemini API documentation: <https://ai.google.dev/gemini-api/docs> — Main Gemini developer documentation.

Gemini API billing: <https://ai.google.dev/gemini-api/docs/billing> — Free and paid tier explanation.

Streamlit installation: <https://docs.streamlit.io/get-started/installation/command-line> — Official venv/pip installation workflow.

Streamlit run guide: <https://docs.streamlit.io/develop/concepts/architecture/run-your-app> — How streamlit run starts a local server.

VS Code Python tutorial: <https://code.visualstudio.com/docs/python/python-tutorial> — Workspaces, Python extension and virtual environments.

Python downloads: <https://www.python.org/downloads/windows/> — Official Windows Python installation.

MLH standard hackathon rules: <https://github.com/MLH/mlh-policies/blob/main/standard-hackathon-rules.md> — General MLH competition rules.

Final beginner checklist

- ☐ I can explain the human problem in one sentence.
- ☐ I know who the intended user is.
- ☐ Python and pip work.
- ☐ VS Code is open on the correct project folder.
- ☐ My .venv is active.
- ☐ requirements.txt installs successfully.
- ☐ streamlit run app.py opens my application.
- ☐ My Gemini API key works and is not inside public source code.
- ☐ I can point to the exact place where my app uses the Gemini API.
- ☐ The core demo works end to end.
- ☐ The app handles a missing key and empty input without crashing.
- ☐ I know what happens if Gemini returns an API error.
- ☐ My GitHub repository does not contain .env, .venv or a real key.
- ☐ My README explains how to run the project.
- ☐ My team has rehearsed the demonstration.
- ☐ We know the OrganizerHQ submission deadline announced by organisers.

Source and currency note

This guide was prepared for Hack for Humanity Harare 2026 using the participant briefing supplied for 19 September 2026 and current official documentation from Google AI for Developers, Streamlit, Python and Microsoft VS Code. Cloud interfaces, model names, quotas and billing options can change; where a screen differs, use the official documentation links above and the organiser's live instructions.

Hackathon-specific facts reflected here include the Best Use of the Google Gemini API challenge, the requirement for meaningful API integration, the expectation to show the human problem and safeguards, the equal MLH judging dimensions, and the requirement to submit through OrganizerHQ.